Day 11 - Working With Real Data, and an Introduction to TypeScript

Day 11 — Working With Real Data, and an Introduction to TypeScript

Objectives

- Use the `fetch` API to make real HTTP requests and work with JSON from a web API
- Understand npm packages in more depth, now that you have a real reason to install one
- Understand *why* TypeScript exists, and set up your first TypeScript file
- Learn TypeScript's basic type annotations, and how to compile TypeScript into plain JavaScript

`fetch` — making HTTP requests

You met JSON on Day 5. Today you'll fetch real JSON from across the internet. Both modern browsers and modern Node.js (v18+, which you have) include a built-in `fetch` function for making HTTP requests — no separate package needed:

```
async function getUser(id) {
    const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

    if (!response.ok) {
        throw new Error(`Request failed with status ${response.status}`);
    }

    const data = await response.json();   // parses the response body as JSON -- itself returns a Pr
    return data;
}
```

`fetch(url)` returns a Promise that resolves once the server has responded (note: it resolves even for error responses like 404 — `response.ok` is how you check whether the status code indicates success, similar in spirit to what you might do in other languages). `response.json()` reads and parses the response body, and is itself asynchronous (since reading the full response body can take a moment), so it needs its own `await` too.

Real code should always account for the request possibly failing entirely — no internet connection, a timeout, a DNS failure — using `try`/`catch` around the whole thing:

```
async function getUser(id) {
    try {
        const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
        if (!response.ok) {
            throw new Error(`Request failed with status ${response.status}`);
        }
        return await response.json();
    } catch (error) {
        console.log(`Could not fetch user: ${error.message}`);
```

```
        return null;
    }
}
```

Why TypeScript exists

You've now spent ten full days seeing what plain JavaScript is capable of — and also seeing, repeatedly, how easy it is to accidentally misuse a value's type without JavaScript complaining until something goes wrong at runtime (Day 1's `typeof`/coercion surprises, Day 3's silent `undefined` from an out-of-range array index, and so on). **TypeScript** adds a layer of optional type annotations on top of ordinary JavaScript, plus a separate tool (the **TypeScript compiler**, `tsc`) that checks those annotations *before* your code ever runs, catching an entire category of mistakes ahead of time — the exact same problem, and the exact same solution, discussed in the Python track's Day 12 regarding Python's type hints, if you've done that track — except in TypeScript's case, the checking is a much more central, expected part of daily work, not an optional add-on.

Crucially, TypeScript code doesn't run directly — it gets **compiled** (translated) into plain JavaScript first, which is what actually runs in Node or a browser. This is a genuinely new idea if everything you've done so far has been "write code, run code directly" — today you'll see the extra compile step for the first time.

Setting up TypeScript

```
npm install --save-dev typescript
npx tsc --init
```

`npm install --save-dev typescript` installs the TypeScript compiler as a **dev dependency** — a package needed only for *developing* your project (compiling/checking your code), not needed by the finished, running program itself (recall this same "regular dependency vs. dev dependency" distinction from the Python track's Day 13, if you've done it, or just take it as a new, TypeScript-specific idea here). `npx` runs a locally-installed package's command-line tool without needing to install it globally first. `tsc --init` creates a `tsconfig.json` file — TypeScript's configuration file, telling the compiler things like which JavaScript version to target and which files to include; you'll look at this in more depth on Day 13.

A TypeScript file ends in `.ts` instead of `.js`. To compile and run one:

```
npx tsc hello.ts        # compiles hello.ts into hello.js
node hello.js               # runs the resulting plain JavaScript, exactly like any other Node scri
```

In practice, most projects use a tool (like `ts-node`, or a bundler) to compile and run in one step during development — you'll set this up properly on Day 13; for today, the two-step compile-then-run process above is worth doing manually at least once, so you understand exactly what's happening underneath.

Basic type annotations

```
let age: number = 25;
let name: string = "Ana";
let isActive: boolean = true;

function add(a: number, b: number): number {
    return a + b;
}
```

The `: number`, `: string`, `: boolean` after a variable name (or after a function parameter, or after a function's closing parenthesis, describing its return type) are **type annotations** — they tell the TypeScript compiler exactly what type of value is expected there. Try deliberately breaking one to see the compiler catch it:

```
let age: number = "twenty-five";   // Type 'string' is not assignable to type 'number'.
```

This exact mistake would have run without complaint in plain JavaScript (dynamic typing, from Day 1) — TypeScript catches it immediately, at compile time, before the code ever runs.

Arrays and objects in TypeScript

```
let numbers: number[] = [1, 2, 3];       // an array of numbers
let names: string[] = ["Ana", "Bo"];        // an array of strings

function greet(person: { name: string; age: number }): string {
    return `${person.name} is ${person.age}`;
}
```

That last example annotates the *shape* of an object directly inline — but writing this out every time gets repetitive fast, which is exactly what tomorrow's lesson (interfaces) solves.

Type inference — you don't always need to write the type out

TypeScript is often smart enough to figure out (infer) a variable's type automatically from its initial value, without you writing an annotation at all:

```
let age = 25;         // TypeScript infers this is a `number`, automatically -- no annotation neede
age = "twenty-five";      // still an error! TypeScript remembers the inferred type and enforces it
```

A widely-followed convention: let TypeScript infer types for simple local variables whose value is obvious from context (like `age = 25` above), and write explicit annotations mainly for function parameters and return types, and for anything whose type genuinely isn't obvious just from looking at the initial value.

`any` — the type that turns off type-checking (avoid it)

TypeScript has an escape hatch, `any`, which tells the compiler "don't check this value's type at all — treat it like plain JavaScript":

```
let anything: any = 5;
anything = "now a string";    // no error -- `any` disables type-checking entirely for this variable
```

Avoid `any` whenever you can. Using it defeats the entire purpose of using TypeScript in the first place for that particular value — it's occasionally necessary (working with a genuinely dynamic, unpredictable piece of data, or gradually converting an old JavaScript codebase to TypeScript file by file), but reaching for `any` as a quick way to make a compiler error go away, rather than actually fixing the underlying type mismatch, is a habit worth avoiding from day one.

Exercises

This day's exercises are in TypeScript. Open `exercises.ts`, fill in each `// TODO`, then compile and run it:

```
npx tsc exercises.ts
node exercises.js
```

If the compiler reports type errors, that's exactly it doing its job — read the message, fix the actual code (not just the type annotation), and recompile.